FundFlow — Reconciliation & Scheduler

This document explains how **operational posting**, **reconciliation**, the **scheduler**, and **system jobs** fit together. It complements the money-movement detail in fund-flow-dynamics.md.

One-page summaries: accountants · admins

Related manuals: manual-accountant.md · manual-administrator.md

---

Table of contents

---

1. Big picture — three layers

```
flowchart TB
    subgraph ops["Operational layer (admin / member actions)"]
        DEP["Deposit accept"]
        CONTRIB["Contribution post / apply"]
        LOAN["Loan disburse / repay / EMI"]
        CASHOUT["Cash-out approve"]
        BANK["Bank CSV import + match"]
        LEG["Legacy migration import"]
    end

    subgraph ledger["Ledger layer"]
        ACCT["AccountingService<br/>member + master mirrors"]
        TXN["Transaction rows"]
        OBS["TransactionObserver"]
    end

    subgraph recon["Reconciliation layer"]
        RT["Realtime checks<br/>(on each Transaction)"]
        SNAP["fund:reconcile<br/>audit snapshots"]
        NIGHT["fund:nightly-reconciliation<br/>exceptions + auto-resolve"]
        GATE["BatchPostingGate<br/>halt batch jobs if critical"]
        EXC["ReconciliationException queue"]
    end

    subgraph sched["Scheduler (routes/console.php)"]
        CRON["Laravel schedule:run<br/>per tenant DB"]
        JOBS["System → Jobs UI<br/>manual re-run"]
```

```
    end

    ops --> ACCT --> TXN --> OBS --> RT
    RT --> EXC
    NIGHT --> EXC
    NIGHT --> GATE
    CRON --> SNAP
    CRON --> NIGHT
    CRON --> BATCH["Batch commands<br/>contributions / loans / bank"]
    BATCH --> ACCT
    GATE -.->|blocks| BATCH
    JOBS --> CRON
    LEG --> ACCT
```

Key idea: Operations **post intent** to the ledger immediately. Reconciliation **verifies** that intent (realtime + nightly). The scheduler **drives recurring collection and checks** on a calendar. They are separate steps — bank clearance, for example, does not re-post cash when you match a line.

Layer	What it does	When it runs
Operational	Admin/member actions (accept deposit, apply contribution, etc.)	On demand
Ledger	Mirrored journal entries via `AccountingService`	Same transaction as the action
Realtime recon	Lightweight guards on each `Transaction`	Immediately after post
Snapshots	Historical audit report stored in `reconciliation_snapshots`	Daily 06:20, monthly 2nd
Nightly batch	Full domain sweeps, auto-resolve, halt gate	Daily 06:30
Scheduler	Recurring collection, fees, bank match, statements	Calendar in `routes/console.php`

---

2. Realtime path on every ledger post

```
sequenceDiagram
    participant Op as Operational service
    participant Acct as AccountingService
    participant Txn as Transaction
    participant Obs as TransactionObserver
    participant Recon as ReconciliationService
    participant Coll as ContributionCollectionCycleService
    participant EMI as LoanInstallmentCollectionService

    Op->>Acct: credit/debit with mirrors
    Acct->>Txn: create transaction line(s)
    Txn->>Obs: created event
    Obs->>Recon: onTransactionPosted()

    Recon->>Recon: audit log TRANSACTION_POSTED
    Recon->>Recon: journal balanced? (UNBALANCED_ENTRY)
    Recon->>Recon: late-fee exemption rules
    Recon->>Recon: member cash/fund drift (MEMBER_*_DRIFT)
```

```
Recon->>Recon: master pool drift (MASTER_*_POOL_DRIFT)

Note over Acct,EMI: Skipped during master pool mirroring<br/>AccountingService::masterPoolMirrorInP

Acct->>Coll: onMemberCashIncreased (when cash credited)
Coll->>EMI: try contribution + EMI from available cash
```

Realtime checks (`ReconciliationService::onTransactionPosted()`):

Check	Exception code	Severity
Journal DR = CR for reference	`UNBALANCED_ENTRY`	critical
Late-fee exemption consistency	varies	varies
Member component formula vs stored balance	`MEMBER_CASH_DRIFT`, `MEMBER_FUND_DRIFT`	high
Master cash/fund vs Σ members	`MASTER_CASH_POOL_DRIFT`, `MASTER_FUND_POOL_DRIFT`	high

Skipped when:

- `ReconciliationService::realtimeChecksSuspended()` is active
- `AccountingService::masterPoolMirrorInProgress()` — paired master/member legs post in one logical operation

Heavy domain sweeps (contributions past window, bank uncleared pipeline, EMI state) run in the **nightly batch**, not on every transaction.

---

3. Daily scheduler timeline

All scheduled tenant commands use `TenantAwareScheduledCommand` — `schedule:run` executes each command **once per tenant database**.

Source: `routes/console.php` · Registry: `App\Support\ScheduledJobRegistry`

Time	Command	Role	Halt-sensitive
06:00	`fund:assert-master-invariants`	Quick gate: master cash/fund = Σ members	No
06:20	`fund:reconcile --daily`	Audit snapshot for yesterday	No
06:30	`fund:nightly-reconciliation`	Control layer: exceptions, auto-resolve, halt gate	No
07:00	`loans:check-defaults`	Delinquency / guarantor / auto-transfer	No

Time	Command	Role	Halt-sensitive
07:15	`contributions:apply-late-fees`	Tier-aged contribution + EMI late fees	**Yes**
07:30	`delinquency:send-digest`	Admin digest notification	No
08:00	`bank:auto-match`	Match imported bank lines to uncleared postings	**Yes**
every minute	`announcements:dispatch-scheduled`	Send due scheduled member announcements	No

```
gantt
    title Typical daily reconciliation and ops window
    dateFormat HH:mm
    axisFormat %H:%M

    section Pool checks
    assert-master-invariants     :06:00, 10m
    reconcile daily snapshot     :06:20, 10m
    nightly-reconciliation       :06:30, 30m

    section Collections and risk
    loan defaults check          :07:00, 15m
    apply late fees              :07:15, 15m
    delinquency digest           :07:30, 10m
    bank auto-match              :08:00, 20m
```

Manual run: Audit & System → Jobs — same commands, subject to `BatchPostingGate` for halt-sensitive entries.

---

4. Monthly collection cycle

Day	Time	Command	Role	Halt-sensitive
1st	08:00	`contributions:init-cycle`	Create pending contribution rows	**Yes**
1st	08:00	`loans:send-due-notifications`	EMI due notifications	No
1st	09:00	`contributions:notify`	Contribution due notifications	No
2nd	06:30	`fund:reconcile --monthly`	Previous month audit snapshot	No
3rd	08:00	`statements:generate --notify`	Monthly statements + notify	No

Day	Time	Command	Role	Halt-sensitive
5th	09:00	`contributions:apply`	Batch debit cash → credit fund	**Yes**
6th	00:30	`contributions:close-window`	Unpaid → overdue	**Yes**
6th	00:45	`loans:close-emi-window`	Unpaid installments → overdue	**Yes**
6th	06:00	`loans:apply-repayments`	Batch EMI collection from cash	**Yes**

```
flowchart LR
    subgraph d1["Day 1"]
        INIT["contributions:init-cycle<br/>pending rows"]
        NOTIFY["contributions:notify"]
        LOAN_N["loans:send-due-notifications"]
    end

    subgraph d5["Day 5"]
        APPLY["contributions:apply<br/>DR cash → CR fund"]
    end

    subgraph d6["Day 6"]
        EMI["loans:apply-repayments<br/>batch EMI from cash"]
        CLOSE_C["contributions:close-window<br/>→ OVERDUE"]
        CLOSE_E["loans:close-emi-window"]
    end

    subgraph daily["Daily (throughout month)"]
        FEES["contributions:apply-late-fees"]
        RT_COLL["Realtime: onMemberCashIncreased<br/>when deposits / imports credit cash"]
    end

    INIT --> NOTIFY
    NOTIFY --> APPLY
    APPLY --> CLOSE_C
    CLOSE_C --> FEES
    LOAN_N --> EMI
    EMI --> CLOSE_E
    RT_COLL -.->|can collect early| APPLY
```

Between scheduled runs: any cash credit (deposit accept, bank import post-to-member, loan disbursement cash leg) triggers `ContributionCollectionCycleService::onMemberCashIncreased()` and `LoanInstallmentCollectionService` — members are not limited to Day 5/6 batch only.

---

5. Nightly reconciliation batch

`fund:nightly-reconciliation` → `ReconciliationService::runNightlyBatch()`:

```
flowchart TD
    START["NIGHTLY_RECON_START"] --> CLEAR["Clear exception queue<br/>(fresh sweep)"]
```

```
CLEAR --> PRE["Master balanced?<br/>tiny drift → rounding suspense"]
PRE -->|critical fail| HALT["BatchPostingGate HALT<br/>stop halt-sensitive jobs"]
PRE -->|ok| DOMAIN

subgraph DOMAIN["Domain sweeps"]
    C["reconcileContributions"]
    L["reconcileLoansAndEmi"]
    F["reconcileFundTiers"]
    B["reconcileBankClearing"]
    LF["reconcileLateFees"]
    M["reconcileMemberInvariants"]
end

DOMAIN --> AUTO["attemptAutoResolve<br/>open exceptions"]
AUTO --> POST["Re-assert master balanced"]
POST --> DONE["NIGHTLY_RECON_COMPLETE"]

HALT --> EXC["ReconciliationException<br/>Finance → Reconciliation"]
DOMAIN --> EXC
```

Sweep	Typical exceptions raised
Master pre-check	`MASTER_IMBALANCE_UNRESOLVED` (critical → halt)
Contributions	`PENDING_PAST_WINDOW_CLOSE`, `COLLECTED_WITHOUT_POST`, tier mismatches
Loans / EMI	`EMI_COLLECTED_LEDGER_MISSING`, `ACTIVE_BEFORE_FULL_DISBURSE`, schedule state
Fund tiers	Tier / fund allocation consistency
Bank clearing	`RECON_UNMATCHED_BANK_LINE`, `UNMATCHED_CASH_ENTRY`, `STALE_PENDING`
Late fees	`FEE_WRONG_TIER`, `FEE_INCOME_DRIFT`
Member invariants	`MEMBER_CASH_DRIFT`, `MEMBER_FUND_DRIFT`

After domain sweeps, **auto-resolve** attempts fix simple cases. A **post-batch** master balance check may halt again if drift remains critical.

---

6. Audit snapshots vs exception queue

These are **different products** — do not confuse them.

Mechanism	Command	Output	Use
Exception queue	`fund:nightly-reconciliation`	Live `reconciliation_exceptions` rows	Active remediation in admin UI
Audit snapshot	`fund:reconcile --daily` / `--monthly`	`reconciliation_snapshots` + PDF metrics	Historical audit trail for accountants
Point-in-time audit	`fund:reconcile --realtime`	Report only (`--no-store` skips save)	Ad-hoc investigation

Snapshots include ledger mismatch counts, unposted bank rows, open exception counts, and coverage matrices — useful for month-end packs, not for replacing the live queue.

---

7. Batch posting gate

```
flowchart LR
    NIGHT["nightly-reconciliation<br/>MASTER_IMBALANCE critical"] --> GATE["BatchPostingGate<br/>halt =
    GATE --> BLOCK["Blocks halt-sensitive jobs"]

    BLOCK --> J1["contributions:init-cycle"]
    BLOCK --> J2["contributions:apply"]
    BLOCK --> J3["contributions:close-window"]
    BLOCK --> J4["loans:close-emi-window"]
    BLOCK --> J5["loans:apply-repayments"]
    BLOCK --> J6["bank:auto-match"]
    BLOCK --> J7["contributions:apply-late-fees"]

    ADMIN["Resolve drift + recon clears"] --> GATE2["gate cleared"]
    GATE2 --> OK["Batch jobs allowed again"]
```

Triggers halt:

- `MASTER_IMBALANCE_UNRESOLVED` critical exception (open or raised during nightly batch)
- Manual halt via system settings (`batch_posting_halted`)

Enforcement:

- `ScheduledJobRegistry` — `halt_sensitive: true` on affected jobs
- `SystemJobRunnerService` — blocks manual runs from Jobs UI
- `EnsuresBatchPostingAllowed` trait — blocks Artisan command execution

Recovery:

1. Fix underlying master/member pool drift.
2. Resolve or write off critical exceptions.
3. Re-run `fund:nightly-reconciliation` (clears gate on success), or clear halt in settings after verification.

Not blocked: deposit accept, manual corrections, snapshot commands, `fund:assert-master-invariants`, notifications, delinquency checks.

---

8. Operational flows through the lifecycle

```
flowchart TB
    subgraph inflow["Cash in"]
        D1["Deposit accept"] --> MC["CR master cash + CR member cash"]
        D2["Bank import"] --> MB["Master bank line"]
        MB --> MIRROR["Mirror → master cash"]
        MIRROR --> MC
        MC --> UNC["Uncleared BankTransaction"]
        UNC --> MATCH["Admin match / bank:auto-match"]
    end

    subgraph use["Cash used"]
        MC --> CON["Contribution collection<br/>DR cash + DR master cash mirror"]
        CON --> MF["CR member fund + CR master fund mirror"]
        MC --> EMI2["EMI / loan repayment<br/>DR cash → fund + loan principal"]
    end
```

```
subgraph outflow["Cash out"]
    MC --> CO["Cash-out approve<br/>DR member + master cash"]
    CO --> UNC2["Uncleared bank payout line"]
end

subgraph loan["Loan fund leg"]
    MF --> DISB["Disbursement<br/>DR member+master fund mirror"]
    DISB --> MC2["CR member+master cash mirror"]
end

MC --> OBS2["Every leg → Transaction → realtime recon"]
MF --> OBS2
```

Every arrow that creates a `Transaction` feeds the realtime reconciliation path (section 2). Bank **match/clear** updates linkage — it does **not** post additional cash legs when done correctly.

---

9. System jobs UI

Location: Audit & System → Jobs

Feature	Behaviour
Job list	Mirrors `ScheduledJobRegistry::all()`
Manual run	`SystemJobRunnerService::run()` — records `SystemJobRun`
Halt badge	Shown when `BatchPostingGate::isHalted()`
History	Per-run exit code, duration, output snippet

Scheduled cron and manual runs use the **same Artisan commands** — only the trigger differs (`SystemJobRun::TRIGGER_SCHEDULED` vs `TRIGGER_MANUAL`).

---

10. Async queue jobs (not on cron)

Job	Trigger	Purpose
`RunLegacyMigrationPaymentsJob`	Legacy Migration UI	Classify + import historical payments
`ClassifyLegacyPaymentsJob`	Legacy Migration UI	Payment classification only
Tenant provisioning jobs	Central app	Cache dirs, tenant setup

Legacy migration runs **outside** the nightly scheduler. After import, operators may run repair commands (`legacy:repair-excess-loan-repayments`, `accounting:rebuild-balances`, etc.) before relying on reconciliation results.

---

11. Summary

FundFlow separates **posting** (what happened economically), **clearance** (does the bank agree?), and **reconciliation** (do the books still obey pool rules?).

1. Day-to-day actions post mirrored ledger entries immediately.
2. Each `Transaction` triggers lightweight realtime checks.
3. Every morning the scheduler runs pool assertions, stores an audit snapshot, then runs a full exception sweep that can halt batch collection jobs if the master pool is broken.
4. Monthly commands open and close contribution/EMI windows; between those dates, cash credits still drive automatic collection via `onMemberCashIncreased`.
5. Admins can re-run any scheduled job from **System → Jobs**, subject to the batch halt gate.

References

Resource	Path
Scheduler definition	`routes/console.php`
Job registry	`app/Support/ScheduledJobRegistry.php`
Nightly batch	`app/Services/ReconciliationService.php`
Halt gate	`app/Support/BatchPostingGate.php`
Manual job runner	`app/Services/SystemJobRunnerService.php`
Mirror rules	`.cursor/rules/accounting-master-member-sync.mdc`
Accountant manual	`docs/manual-accountant.md`
Administrator manual	`docs/manual-administrator.md`